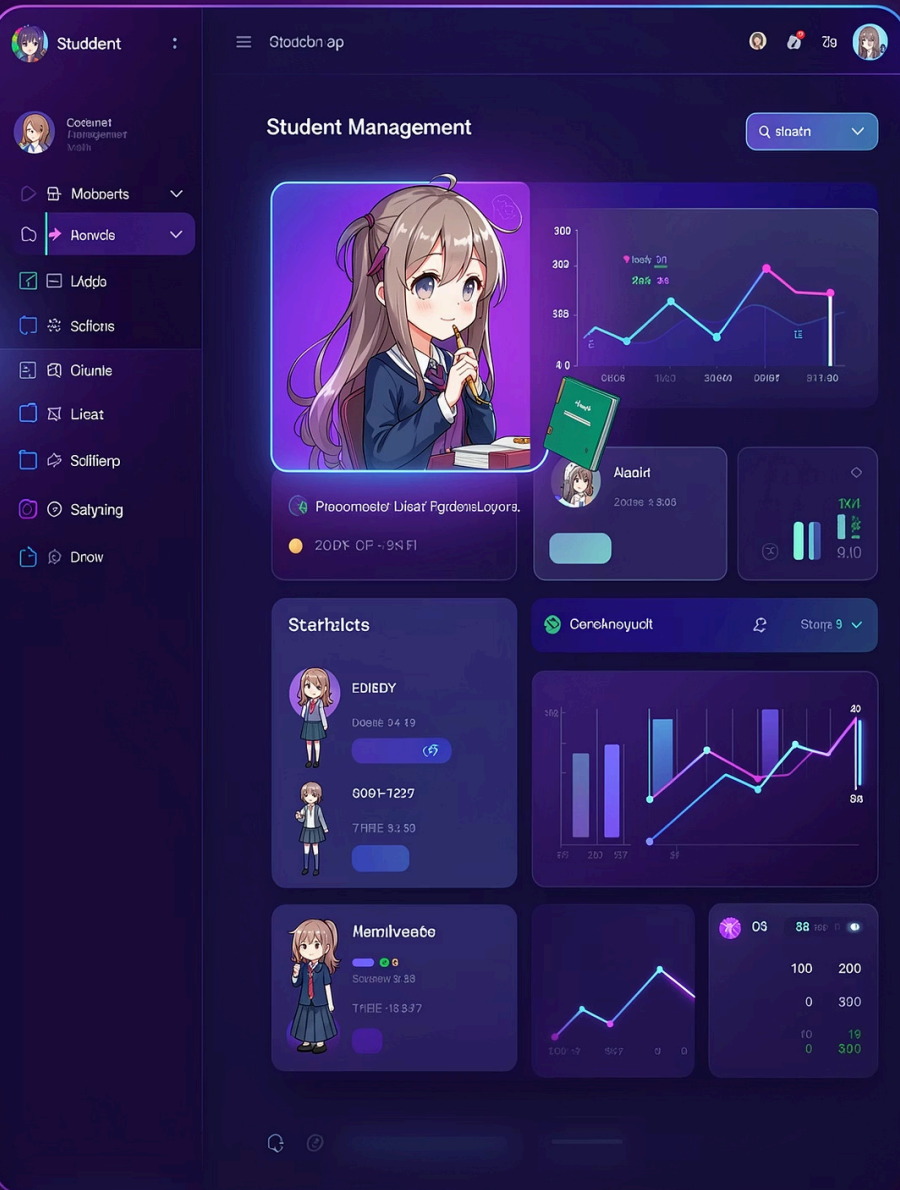




Sistema Completo de Gerenciamento de Alunos

Documentação técnica profissional de um sistema desenvolvido em Python com interface gráfica, camada lógica, banco de dados remoto (Firebase via API REST) e persistência local de fallback.

PYTHON

FIREBASE

API REST

CUSTOMTKINTER

Visão Geral do Sistema

O sistema implementa um fluxo completo de aplicação real, cobrindo todas as camadas essenciais de um software profissional. O objetivo principal é gerenciar alunos, permitindo cadastro, consulta, alteração e exclusão, com sincronização entre dados locais e remotos.

Interface Gráfica

Frontend desktop desenvolvido com **customtkinter**, oferecendo uma experiência visual moderna e intuitiva para o usuário final.

Camada Lógica

Processamento de dados e regras de negócio centralizadas, garantindo consistência e integridade em todas as operações do sistema.

Firebase via API REST

Banco de dados remoto acessado por requisições HTTP, permitindo sincronização em tempo real dos dados de alunos.

Persistência Local

Camada de fallback que armazena dados localmente, garantindo funcionamento offline quando o Firebase não está disponível.

Arquitetura do Sistema

O sistema segue o princípio de **separação de responsabilidades**, dividido em 4 módulos independentes. Essa estrutura evita acoplamento entre as camadas e permite manutenção mais fácil e escalabilidade do projeto.

1

dados.py

Camada de armazenamento local. Responsável por persistir a lista de alunos e a soma das médias, servindo como backup quando o Firebase não está disponível.

2

funções.py

Camada de lógica de negócio. Contém todas as regras do sistema, como manipulação de listas, cálculo de média e validação de dados.

3

bancodedados.py

Camada de integração com Firebase. Realiza comunicação remota via API REST usando requisições HTTP para CRUD completo.

4

beta2.py

Camada de interface gráfica. Implementa o frontend desktop com customtkinter, gerenciando janelas, botões e reprodução de mídia.

Módulo bancodedados.py — API REST

Este módulo realiza a comunicação com o Firebase através de requisições HTTP. Demonstra domínio de API REST e manipulação de dados remotos, cobrindo todas as operações de CRUD.

Operações HTTP

- **POST** (requests.post) → cria dados
- **GET** (requests.get) → lê dados
- **PATCH** (requests.patch) → altera dados
- **DELETE** (requests.delete) → remove dados

Funções Principais

- **Criar:** Monta um dicionário com nome, nota e situação, converte para JSON, envia POST para o Firebase, verifica status_code e exibe mensagem de sucesso
- **Apagar:** Faz GET geral, percorre a lista de alunos, compara nomes e, ao encontrar o aluno, executa DELETE
- **Alterar:** Permite modificar campos específicos usando PATCH, trabalhando com listas para múltiplos campos
- **LigacaoComDadosSalvos:** Executa GET, converte JSON em Python, monta lista de dicionários e retorna a lista

Módulo beta2.py — Interface Gráfica

Este arquivo implementa a interface gráfica do sistema usando **customtkinter**, representando o frontend da aplicação. Gerencia janelas, botões e reprodução de mídia, além de coordenar o carregamento inicial dos dados.



Inicialização

Define o tema da aplicação, configura a aparência visual e inicializa o **pygame** para controle de áudio.



Carregamento de Dados

Tenta carregar dados do Firebase ao iniciar. Caso não existam dados remotos, utiliza os dados locais como fallback.



Gerenciamento de Janelas

ChamarJanela cria a janela principal com tamanho e título definidos.

CriarSubJanela gera janelas secundárias utilizando CTKToplevel.



Função Midia

Controla a reprodução de áudio e vídeo na interface, utilizando as bibliotecas **pygame** e **tkvideo**.



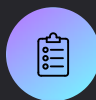
Funções de Interface — beta2.py

A interface gráfica oferece cinco operações principais acessadas por botões, cobrindo o ciclo completo de gerenciamento de alunos. Cada função cria ou atualiza elementos visuais na janela principal.



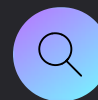
Adicionar

Cria a interface de cadastro de novos alunos, permitindo inserção de nome, nota e situação.



Mostrar

Exibe a lista completa de alunos cadastrados na interface, com todas as informações relevantes.



Consultar

Permite buscar um aluno específico pelo nome, retornando seus dados na interface.



Alterar

Atualiza os dados de um aluno existente, modificando campos específicos conforme necessidade.



Apagar

Remove um aluno do sistema, tanto da interface quanto do banco de dados remoto.

Módulo funções.py — Lógica de Negócio

Responsabilidades

Este módulo é responsável por toda a regra de negócio do sistema, atuando como o núcleo de processamento entre a interface e os dados. Garante a consistência do sistema em todas as operações.

- Manipular a lista de alunos
- Calcular média geral
- Validar dados de entrada
- Atualizar valores globais

Fluxo Típico de Processamento

1 Recebe entrada do usuário

Captura os dados informados pela interface gráfica.

2 Processa os dados

Aplica validações e regras de negócio sobre as informações recebidas.

3 Atualiza a lista

Modifica a lista de alunos com os novos dados processados.

4 Atualiza média geral

Recalcula e atualiza os valores de média do sistema.

Módulo dados.py — Persistência Local

Camada simples de persistência local que garante o funcionamento offline do sistema. Atua como backup quando o Firebase não está disponível, assegurando que os dados dos alunos não sejam perdidos.

O que Armazena

- **Lista de alunos:** Todos os registros cadastrados no sistema, incluindo nome, nota e situação de cada aluno
- **Soma das médias:** Valor acumulado utilizado para cálculo da média geral da turma

Quando é Utilizado

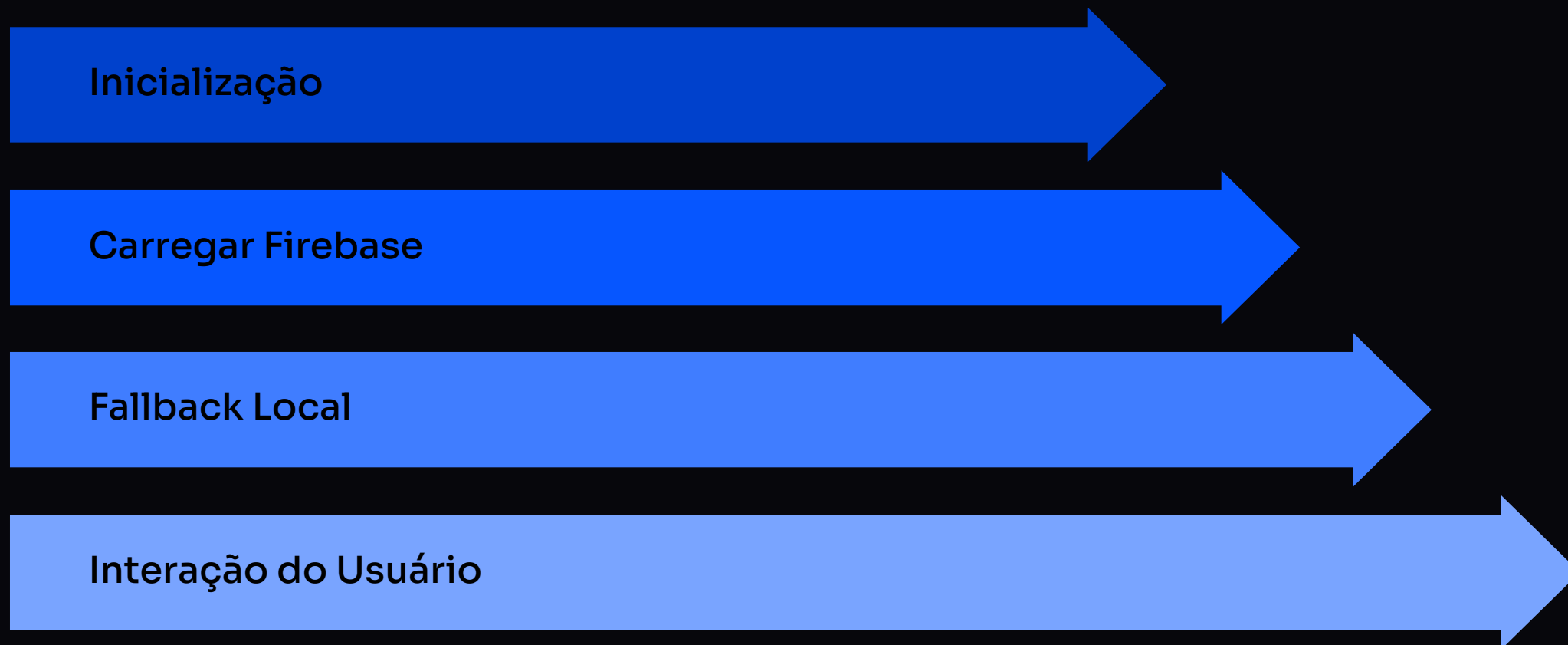
O módulo **dados.py** é acionado quando o Firebase não está disponível. Durante o carregamento inicial, o sistema tenta buscar dados remotos; caso não existam ou haja falha de conexão, recorre aos dados locais armazenados neste módulo.

Importância Estratégica

Garante que o sistema continue operacional mesmo sem conexão com a internet, proporcionando resiliência e confiabilidade à aplicação. Essa camada de fallback é essencial para um fluxo de aplicação real.

Fluxo Completo do Sistema

O fluxo abaixo demonstra a integração completa entre todas as camadas do sistema, desde a inicialização até a sincronização com o Firebase. Esse processo mostra como interface, lógica, banco de dados remoto e persistência local trabalham de forma coordenada.



O sistema inicia pela interface gráfica (**beta2.py**), que tenta carregar dados do Firebase via **bancodedados.py**. Se os dados remotos estiverem vazios ou indisponíveis, utiliza os dados locais armazenados em **dados.py**. O usuário então interage com os botões da interface (Adicionar, Mostrar, Consultar, Alterar, Apagar), e as funções de negócio em **funções.py** processam os dados — validando entradas, manipulando listas e calculando médias. Por fim, os dados são enviados ou atualizados no Firebase, completando o ciclo de sincronização entre todas as camadas.



Conclusão — Projeto para Portfólio Profissional

Este sistema demonstra um **nível profissional de desenvolvimento**, integrando múltiplas tecnologias em uma arquitetura modular e bem estruturada. Trata-se de um excelente projeto para portfólio profissional.

Competências Aplicadas

- Interface gráfica (customtkinter)
- Programação estruturada em Python
- Integração com API REST (Firebase)
- Manipulação de dados JSON
- Arquitetura modular com separação de responsabilidades

Potencial de Expansão

- Migração para plataforma **Web**
- Substituição por banco **SQL**
- Implementação de sistema **multiusuário**

- ✔ O projeto pode ser expandido para web, banco SQL ou sistema multiusuário, consolidando-se como base sólida para aplicações profissionais mais complexas.